

ISMP 3D Anatomy — Arquitectura y Flujo del Sistema

Diagrama de componentes · Estado actual y pendientes

Referencias:



Módulo / Componente



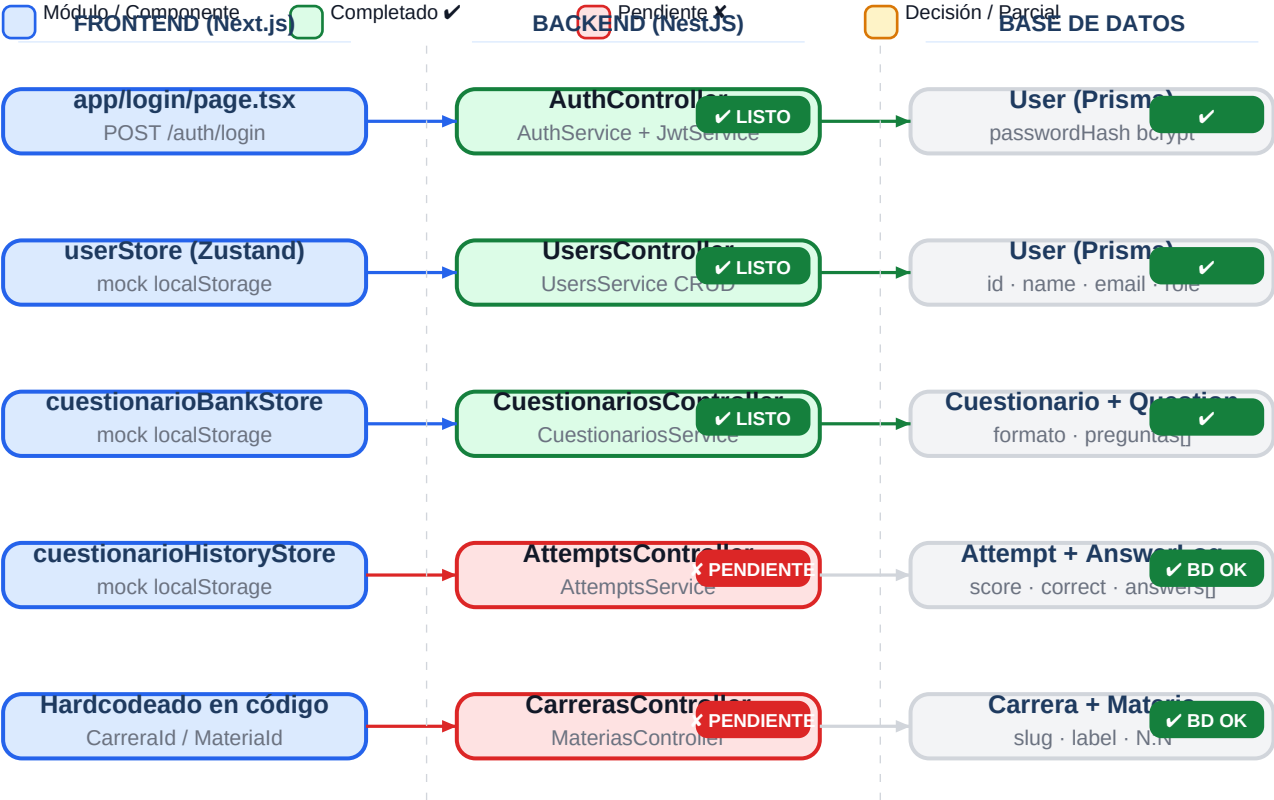
Completado ✓



Pendiente ✗



Decisión / Parcial



Flujo Detallado — Autenticación y Cuestionarios

Secuencia de llamadas con guards y validaciones

1 LOGIN / AUTENTICACIÓN

Usuario ingresa email + password

`AuthController.login(dto)`

NestJS recibe la petición

`prisma.user.findUnique({ email })`

Busca usuario en PostgreSQL

¿Usuario existe?

NO

401 Credenciales incorrectas

SI

`bcrypt.compare(password, hash)`

Verifica contraseña

¿Password correcta?

NO

401 Credenciales incorrectas

SI

`jwtService.signAsync({ sub, email, role })`

Genera JWT

Respuesta: { token, user }

200 OK → Frontend guarda JWT

2 REQUESTS PROTEGIDAS (JWT Guard)

Frontend envía request

Authorization: Bearer <token>

`JwtGuard.canActivate()`

Valida firma y expiración del JWT

¿Token válido?

NO

401 Unauthorized

SI

`RolesGuard.canActivate()`

Verifica rol del JWT vs @Roles()

¿Rol permitido?

NO

403 Forbidden

SI

Controller ejecuta la lógica

Service → Prisma → PostgreSQL

Respuesta 200 OK con datos

JSON al frontend

Roadmap — Lo que falta implementar

Fases ordenadas por prioridad · Conexión Frontend ↔ Backend

1

FASE 1 — Módulo Attempts (Backend)

Crear AttemptsController + AttemptsService + AttemptsModule en NestJS

- POST /cuestionarios/:id/attempts → guarda intento + AnswerLog[]
- GET /cuestionarios/:id/attempts → historial por cuestionario (DOCENTE/ADMIN)
- GET /users/me/attempts → historial del estudiante logueado (JWT requerido)



2

FASE 2 — Conectar Login del Frontend

Reemplazar el mock de userStore por la respuesta real del backend

- Cambiar URL: app/login/page.tsx → POST http://localhost:3001/auth/login
- Guardar el JWT en localStorage al recibir la respuesta
- Crear helper authFetch() que adjunta Authorization: Bearer <token>



3

FASE 3 — Conectar Cuestionarios

Reemplazar cuestionarioBankStore (mock) por llamadas reales al backend

- GET /cuestionarios → reemplaza la lista del store
- GET /cuestionarios/:id → reemplaza el cuestionario individual
- POST/PATCH/DELETE con JWT → desde panel del docente



4

FASE 4 — Conectar Historial de Intentos

Reemplazar cuestionarioHistoryStore (mock) por endpoints reales

- Al terminar quiz: POST /cuestionarios/:id/attempts (con answers[])
- Mis resultados: GET /users/me/attempts
- Panel docente: GET /cuestionarios/:id/attempts



5

FASE 5 — Catálogos Carreras y Materias

Exponer los catálogos de BD y eliminar los valores hardcoded del frontend

- Crear GET /carreras y GET /materias en NestJS
- Reemplazar arrays fijos en el frontend por estas llamadas
- Usar slugs reales como filtros en cuestionarios



6

FASE 6 — Integración con BD del Cliente

Migrar la conexión a la base de datos real del cliente

- Conectarse a la red del cliente (VPN o red local)
- Actualizar DATABASE_URL en .env con credenciales reales
- Ejecutar: npx prisma db pull (si la BD ya existe con tablas)
- Ajustar migraciones y correr npx prisma db push si es BD nueva

Flujo Completo — Estudiante rinde un Cuestionario

Desde el login hasta el registro del intento en la BD

